THE ARAB AMERICAN UNIVERSITY

FACULTY OF INFORMATION TECHNOLOGY

ALGORITHMS AND PROGRAMMING TECHNIQUES (240113020)

Assignment: (10 Marks)

Part 1:

Q1)

Sort the list *E, X, A, M, P, L, E* in alphabetical order by selection sort.

Q2)

How many comparisons (both successful and unsuccessful) will be made by the brute-force algorithm in searching for each of the following patterns in the binary text of one thousand zeros?

a. 00001

b. 10000

c. 01010

Q3)

a. Write pseudocode for a divide-and-conquer algorithm for finding the position of the largest element in an array of *n* numbers.

b. What will be your algorithm's output for arrays with several elements of the largest value?

c. Set up and solve a recurrence relation for the number of key comparisons made by your algorithm.

d. How does this algorithm compare with the brute-force algorithm for this problem?

Q4)

a. Design a brute-force algorithm for computing the value of a polynomial

$p(x) = a_n x^n + a_{n-1} x^{n-1} + \ldots + a_1 x + a_0$

at a given point x_0 and determine its worst-case efficiency class.

b. If the algorithm you designed is in $\Theta(n^2)$, design a linear algorithm for this problem.

c. Is it possible to design an algorithm with a better-than-linear efficiency for this problem?

Part 2:

Implement the following three functions within a single C++ program. Each function should return the **starting index** of the first occurrence of the pattern P in text T, or -1 if not found.

1. **Naive Search (Brute Force):**
    - Compare characters one by one.
    - Shift the pattern by exactly one position after every mismatch.
2. **Knuth-Morris-Pratt (KMP):**
    - Implement the compute LPS (Longest Proper Prefix which is also Suffix) helper function.
    - Use the LPS array to skip unnecessary comparisons.
3. **Boyer-Moore (Simplified):**
    - Implement the **Bad Character Heuristic** using a shift table (precompute the last occurrence of each character).
    - Perform matching from **right to left** within the window.

Using the following data:

- **Text (T):** BACDGABCDAAB
- **Pattern (P):** ABCDA

Provide direct answers for the following:

1. **KMP Table:** Write the resulting LPS array for the pattern P.
2. **Boyer-Moore Table:** List the shift values for characters A, B, C, D in the Bad Character table.
3. **Comparison Count:** How many character-to-character comparisons does the **Brute Force** algorithm perform before reaching the first successful match?
4. If **P** is a string of **m** identical characters (e.g., AAAAA) and **T** is a string of **n** identical characters (e.g., AAAAAAAAAA), which algorithm is the least efficient? Briefly state why.

- **Code:** A .cpp file containing the three algorithms and a main() function that tests them with the strings provided in Part 2.
- **Trace:** A text file or PDF showing the LPS array, the Bad Character table, and your comparison count.

Assignment Instructions

- **Deadline: Thursday، May 21, 2026**, at **11:50 PM**. (Late submissions will not be accepted).
- **Individual Work:** This is a strictly individual assignment. Sharing code or solutions with peers is prohibited.
- **AI Policy:** Use of AI tools (e.g., ChatGPT) to generate solutions is strictly forbidden.
- **Documentation:** Include comments explaining your logic and specify the Time Complexity using **O** notation.

Note: All submissions will be screened for plagiarism. Identical or AI-generated code will result in a grade of **zero** for all involved.

Best of luck!